

Bug-based T^2 : A New Globally Convergent Potential Field Approach to Obstacle Avoidance

Javier Antich and Alberto Ortiz

Department of Mathematics and Computer Science

University of the Balearic Islands

Cra. Valldemossa km. 7'5, 07122 Palma de Mallorca, Spain

{javi.antich, alberto.ortiz}@uib.es

Abstract—Reactive mobile robot navigation based on potential field methods has shown to be a good solution for dealing with both unknown and dynamic environments, where timely responses are required. Unfortunately, the complexity of the tasks which can be successfully carried out is restricted by the inherent shortcomings of the approach such as trapping situations due to local minima, difficulties passing among closely spaced obstacles, oscillations in narrow corridors, etc... The first of the aforementioned limitations was fully overcome in [1] by applying the two new principles of *Traversability* and *Tenacity* in the context of artificial potential fields. As a result, navigation was achieved in very difficult scenarios, even including maze-like environments. In this paper, continuing that previous work, it is intended to step forward by ensuring, whenever possible, that the target point is attained for any mission, setting, at the same time, an upper bound regarding the length of the resultant robot's path.

I. INTRODUCTION

In robotic navigation, potential field methods (PFM) are a well-known solution for dealing with unknown and dynamic scenarios by taking into account the reality of the environment during the robot motion. The characteristic elegance and simplicity of the approach when representing and successfully solving a path-planning problem in real-time explains its extensive application in this field. Nevertheless, substantial shortcomings have been identified as problems inherent to this principle [2]. *Getting stuck in local minima* is the best-known and most often-cited problem with PFMs. In consequence, several obstacle configurations such as the typical U-shaped canyon may lead to undesirable trapping situations. Another drawback associated with this kind of systems is related to the impossibility to go through small openings. Finally, the lack of an oscillation-free motion when the robot navigates among very close obstacles at high speed constituted until recently [3] the last significant problem of PFMs.

In this context, a solution to the first of the above-mentioned problems was given in [1] according to the so-called concepts of *Traversability* and *Tenacity* —or T^2 , in brief. As a result, navigation was achieved in very difficult scenarios, even including maze-like environments. In this paper, continuing that previous work, it is intended to step forward by ensuring, whenever possible, the attainment of the goal point for any mission setting, at the same time, an upper bound concerning the length of the resultant robot's trajectory.

With this purpose, the way how the T^2 principles were originally applied in [1] is considerably simplified by removing the need of remembering past actions/decisions. An emergent global behaviour is thus obtained, which is essentially characterised by the activation of a contour following process after the detection of an obstacle impeding the robot's progress. This process is abandoned/left once the goal can be, at least in appearance, directly reached. In short, the resultant strategy is very efficient from the point of view of the length of the path which is finally generated. Unfortunately, it does not always guarantee the mission completion. The opposite features —longer trajectories and convergence— are, in general, found in one of the most popular families of algorithms for path-planning with incomplete information called *Bug*, initially put forward in [4]. The new navigation strategy *Bug-based T^2* combines the leaving conditions of both approaches to make the most of their corresponding advantages.

The rest of the document is organised as follows: section II briefly introduces the classic potential field approach, while section III describes the main features of *Bug-based T^2* ; some real results are presented in section IV; and, finally, section V concludes the paper by pointing out its major contributions as well as the forthcoming work.

II. FRAMEWORK

The classic potential field approach proposed in [5] constitutes the basic framework for the application of the novel navigation strategy *Bug-based T^2* . It computes the motion of the robot on the basis of two simple behaviours: *GoTo* and *AvoidObstacles*. More precisely, the former generates an attractive force in direction to the goal, while the latter considers obstacles as repulsive surfaces. The robot follows the negative gradient of the resulting potential field towards its minimum, whose position is expected to coincide with the goal point.

III. THE STRATEGY *Bug-based T^2*

The inability to move the robot away from the goal direction in a non-momentary and strategic way is the main cause of the undesirable trapping situations suffered by the reactive control paradigm and, in particular, by the artificial potential fields. Fig. 1(a) shows an example where a robot adopting this last approach is unable to escape from a U-shaped obstacle. A

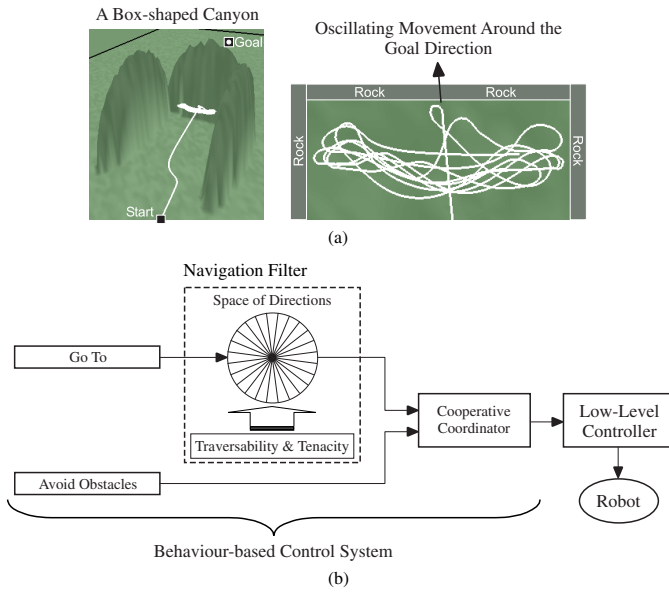


Fig. 1. (a) A typical trapping situation for reactive control systems. (b) Integration of the navigation filter into the classic potential field approach.

solution is given next by applying two concepts, *Traversability* and *Tenacity*, in the context of the so-called navigation filter. As a result, those trapping situations linked to the local minima problem are successfully avoided. It will be also seen that unfortunately this module does not always ensure the completion of the mission. In consequence, some changes are afterwards introduced in order to guarantee the convergence. Finally, notice that the navigation filter is not an isolated control module but a new component of a generic behaviour-based control system. Fig. 1(b) illustrates its integration into the classic potential field approach. The resulting control diagram will be taken as a reference from now on.

A. Fundamentals of the Navigation Filter

The main task of the navigation filter is the appropriate alteration of the direction of the motion vector generated by the *GoTo* behaviour in order to rapidly overcome any obstacle irrespective of its size and shape (see fig. 1(b) again). Such change is carried out according to the *Traversability* and *Tenacity* principles. In short, the first one suggests banning first those directions where an obstacle has been detected and, secondly, choosing an obstacle-free direction close to the *GoTo*'s response, when it has been banned. In this last case, the *tenacity* principle determines, finally, what obstacle-free direction is selected among all the available alternatives. Both principles and their application are deeply described in the following.

1) *The Traversability Principle*: The application of the *traversability* principle requires the division of the space of directions around the robot into K identical angular regions as it is shown in fig. 2(a). These regions are additionally classified as *banned* or *allowed*. Specifically, a region belongs to the former group when at least one obstacle is known to be in

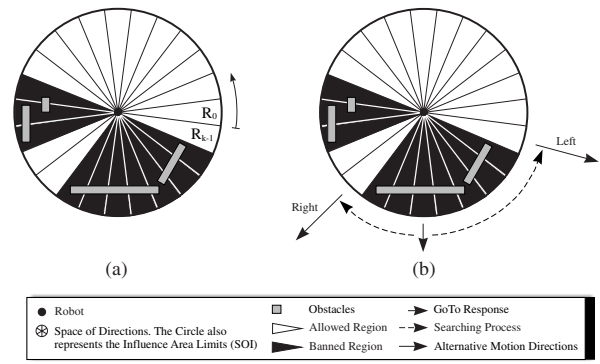


Fig. 2. Exemplification of the implementation of the *traversability* principle: (a) division of the space of directions into K regions, labeling them as *allowed* or *banned*; (b) selection of two obstacle-free motion directions.

the range of directions which consists of. The non-existence of obstacles, on the other hand, characterises the latter group.

Based on the previous information, this principle is intended to forbid the robot's movement in directions where the presence of obstacles has recently been determined, avoiding thus unnecessary and unsuccessful displacements in the task of looking for a path towards the goal point. With this purpose, after receiving the response of the *GoTo* behaviour, its viability is studied according to the above-mentioned premise. Changes are required only if the direction of such motion vector lies in a *banned* region. Otherwise, the navigation filter does not act, adopting as output the same input vector. In such a case, the classic potential field approach is actually used to produce the response of the control system. If, however, the motion vector direction is *banned*, a double searching process starts, which looks, clockwise and counterclockwise, for the first *allowed* region starting from the *GoTo* direction of motion; the two alternative resulting directions are generically labeled as *left* and *right* (see fig. 2(b) for an example). The final decision about choosing one direction or another depends on the *tenacity* principle.

2) *The Tenacity Principle*: Taking fig. 1(a) as an example, the trajectory of a robot which has been trapped can be seen, in a simplified way, as the result of two simple movements: forward and backward. These movements alternate the control of the robot each time the vehicle moves excessively away from the goal direction. The *tenacity* principle precisely tries to give a solution to this oscillating/hesitant behaviour by preventing the robot from drastically changing its direction of motion. In this way, progress is always ensured, removing thus the main cause of any trapping situation. As for the implementation details, remember that two alternative motion directions labeled as *left* and *right* are obtained when the response of the *GoTo* behaviour lies in a *banned* region. Under these circumstances, one of such directions has to be selected as output of the navigation filter. With this aim, the *tenacity* principle is applied by choosing *left* or *right* in coincidence with the last decision made. Despite the obvious simplicity of the concept, the *tenacity* principle has proven fully effective.

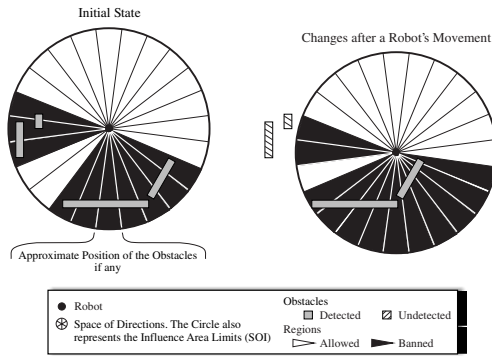


Fig. 3. Memorisation of the obstacle locations while navigating.

3) *Need of Remembering the Obstacles*: Purely reactive systems such as the classic potential field approach react directly to the world as it is sensed, avoiding the need of making intervene any kind of abstract representational knowledge. The sentence “what you see is what you get” faithfully summarises this idea. However, the local information provided by the robot’s sensory equipment may not be enough to solve a given navigation task. Our navigation filter, against this problem, keeps and uses information regarding the obstacles beyond the robot’s immediate sensory range. The approach is, nevertheless, still reactive since no path planning is ever conducted on such information. By way of example, fig. 3 shows how the navigation filter is able to remember the presence of obstacles in directions where, currently, an obstacle-free space is being locally detected. To this end, the approximate location of the obstacles in the environment is memorised. It is important to note, however, that the character of these data is temporary being removed when the robot is sure that the corresponding obstacle has been successfully overcome. Further details about this subject are given later.

B. The Emergent Global Behaviour

The combined application of both the *traversability* and *tenacity* principles results in an emergent global behaviour which can be summarised in the next three points:

1. When the robot is navigating far from obstacles, it heads for the goal following a straight path. During this period of time, the navigation filter remains inactive generating as output the same input motion vector.
2. After the detection of an obstacle, the robot follows its contour in a certain direction. To this end, the response of the *GoTo* behaviour is properly altered according to T^2 . Of special interest is, however, the first time that a change is performed. In such a case, the *tenacity* principle cannot be applied due to the lack of a previous decision so that an additional selection criterion is needed for these particular situations. This first decision determines the direction taken by the robot when following the obstacle boundary. In short, the decision is made on the basis of how the previous obstacle was followed, or lacking this information —first obstacle—, according to a minimum turn criterion.

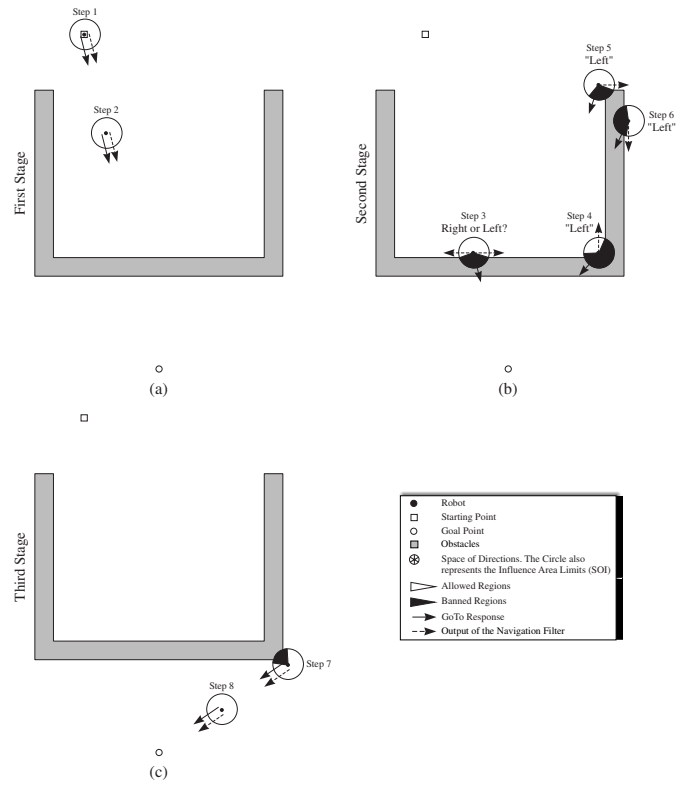


Fig. 4. Escaping from a U-shaped obstacle: (a) direct path to the goal point; (b) following the contour of the obstacle to the robot’s left (note that this direction is defined in step 3 by selecting the option labeled as left based on a minimum turn criterion); (c) reset of the navigation filter returning, afterwards, to (a) where the goal is finally reached.

3. Finally, the robot knows that the obstacle has been overcome when the direction to the goal becomes free of obstacles, that is, not *banned*. At that moment, the filter is reset losing thus all the previously kept information—obstacle positions—, which is no longer necessary for the navigation task.

These stages are sequentially executed in the order specified, as many times as obstacles the robot finds on its way towards the goal point (see fig. 4 for a simple example). On the other hand, before concluding, note that the *AvoidObstacles* behaviour also plays an important role in the final robot’s behaviour. More precisely, it helps to maintain a reasonable distance between the robot and the contour of an obstacle when it is being followed.

C. Some Relevant Considerations

Some more specific details about the navigation filter can be found next:

- First of all, consider the situation illustrated in fig. 5(a). As can be observed, after step 5, the robot is not allowed to head for the goal point because it is known there are obstacles in that direction, although they are really far away. Consequently, this mission is never successfully completed. In order to solve this problem, the information recorded by the navigation filter is partially

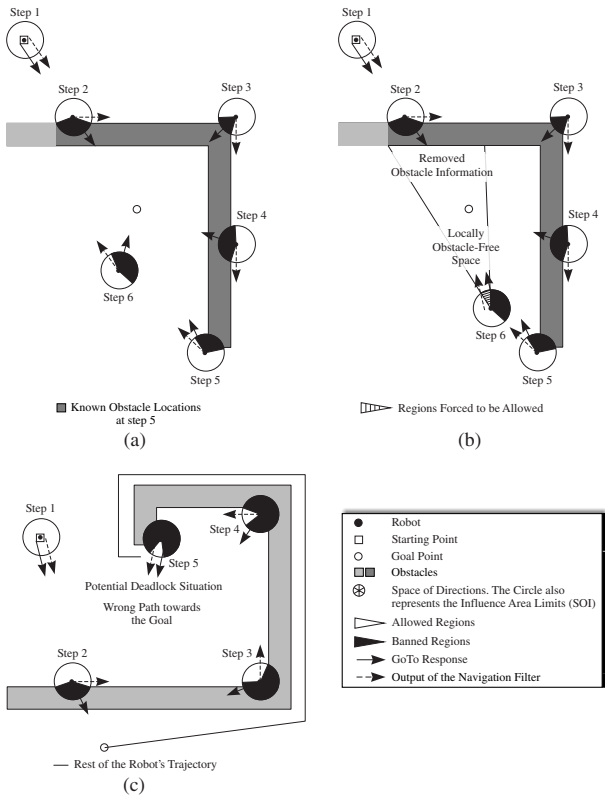


Fig. 5. Extra capabilities of the navigation filter (for each mission, the most significant steps are outlined): in (b), the obstacle information which does not agree with the local environment reality is removed in a controlled and strategic way, solving the problem depicted in (a); in (c), concavities are identified and overcome, making the robot face the goal.

contrasted with the immediate reality of the environment in the next way: after applying the *traversability* and the *tenacity* principles, the *banned* region next to the one chosen is checked for feasibility so that its state is changed to *allowed* if no obstacles are detected in its vicinity for the corresponding range of directions. In this way, it is intended to remove that information—obstacle locations—which is no longer useful for the navigation task. Fig. 5(b) depicts the trajectory of the robot after incorporating this control logic for the mission initially taken as example.

- Secondly, notice that the navigation filter is able to immediately detect potential deadlock situations, mainly due to environment concavities, which may result in a cyclic behaviour. These are characterised by a space of directions fully, or almost fully, *banned*. In this kind of cases, contrary to other strategies, the robot *tenaciously* continues following the boundary of the obstacle rejecting an apparently correct but, in general, wrong path to the goal. Fig. 5(c) presents an example of this situation.

Finally, refer to [6] for a complete description of the operation of the navigation filter in pseudocode.

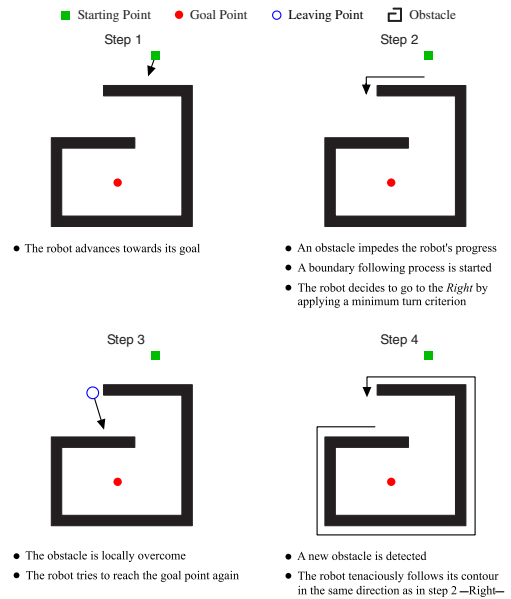


Fig. 6. A mission not solved by the navigation filter. Note that after step 4, step 3 is executed again, generating thus a cyclic behaviour. The robot's trajectory has been computed by hand according to the known three stages of the strategy (refer to section III-B).

D. Achieving Convergence

As was already said, the successful mission completion cannot be always guaranteed by means of, only, the navigation filter—that is, the T^2 principles. Fig. 6 shows, precisely, an example confirming this fact. As can be observed, the robot is not able to reach the goal point since it is immersed in a cyclic behaviour around the G-shaped obstacle. In the following, an efficient solution to the problem is given by slightly modifying the way how the robot decides to leave the contour of the obstacles.

At this moment, the boundary of an obstacle is essentially left when the direction to the goal becomes *allowed*. Note that, from a geometric point of view, such circumstance exclusively arises in situations where the robot faces the goal point while following the obstacle contour. These points of leaving, taking into account the local information which is really managed by the strategy, correspond to the most promising and closest places from where the robot would be supposedly capable of completing the mission through a direct path. This advantage, however, contrasts with the lack of convergence which is derived from such decisions.

Regarding the leaving of the contour following process, other interesting criteria can be found in the related literature. More precisely, the strategy *Bug2* [4] suggests that the robot stops following the boundary of an obstacle when its trajectory cuts the straight segment joining the starting and the target points, also named *Main Line*—or *M-line*, in brief. As a result, this strategy guarantees the goal achievement at the expense of, generally, longer paths. Fig. 7(a) and (b) compare, from the viewpoint of the resultant robot's trajectory, the leaving conditions applied by respectively the T^2 and *Bug2* approaches.

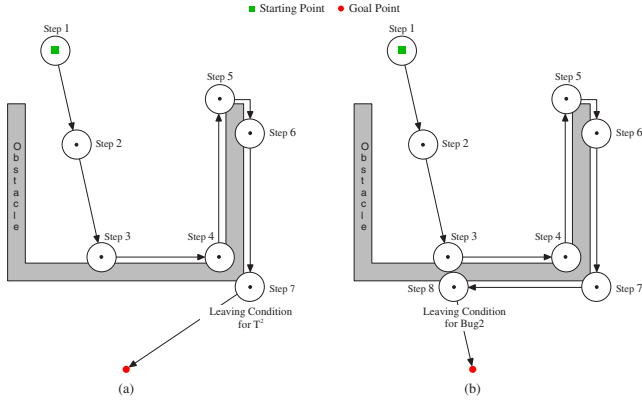


Fig. 7. Comparison of the leaving conditions linked to the (a) T^2 and (b) Bug2 strategies in a simple scenario.

Both advantages, a good path length performance as well as convergence, can be really attained by combining and slightly altering the previous conditions in a proper way. On the one hand, the leaving conditions for T^2 (C1) and Bug2 (C2) are defined as follows:

- C1) The navigation filter indicates there is a free-obstacle path towards the goal. Moreover, this must be the first time the robot leaves the obstacle at approximately the same position.
- C2) The robot's trajectory cuts the M -line. Additionally, the distance from the robot to the goal point has to be shorter than the one associated with the last time this condition was satisfied. Initially, this distance is initialised to the M -line's length.

On the other hand, either C1 or C2 must be met for the leaving to occur. At this point, notice that, contrary to the Bug2 algorithm, the M -line concept is not static for Bug-based T^2 . Specifically, each time the leaving condition C1 is satisfied, the M -line is modified by considering as new starting point the current robot position¹. This fact, together with the limited number of times that condition C1 can be fulfilled², allow proving the convergence of Bug-based T^2 . The formal proof can be found in the appendix.

Finally, an upper bound for the length of the path P generated by Bug-based T^2 has been obtained for a restricted situation: *assume a navigation environment composed by obstacles of arbitrary shape —convex and non-convex— where a straight line starting from the goal point can be drawn without intersecting any obstacle, then the worst case for the P 's length is given by the expression*

$$P < \sum_{i=1}^n p_i + \delta, \quad (1)$$

where n represents the number of obstacles, p_i denotes the perimeter of the i th obstacle and, lastly, δ refers to the length of the paths required to move the robot from the starting point

¹In such a case, the distance used by condition C2 is also reinitialised.

²This assertion is based on the assumption that there is a finite number of obstacles in the navigation environment.

to the first obstacle, among the different obstacles and from the last obstacle to the goal. In short, expression 1 summarizes the following fact: in the aforementioned obstacles configuration, a piece of the boundary of an obstacle is covered by the robot on its way to the goal point just once at the most. Moreover, a contour is never fully covered according to the inequality of expression 1.

IV. EXPERIMENTAL RESULTS

To show the usefulness of Bug-based T^2 , the following presents an experiment involving a Pioneer 3DX robot equipped with 8 ultrasonic sensors for obstacle detection navigating within the scenario depicted in fig. 8(a) and (b). As can be observed, the robot was placed inside a G-shaped obstacle being the goal located outside of it. This mission essentially corresponds to that one not solved by the navigation filter on the basis of, exclusively, the T^2 principles (remember fig. 6). The results obtained can be found in fig. 8(a). (For a better understanding, the robot's trajectory is step-by-step drawn and described in fig. 8(c).)

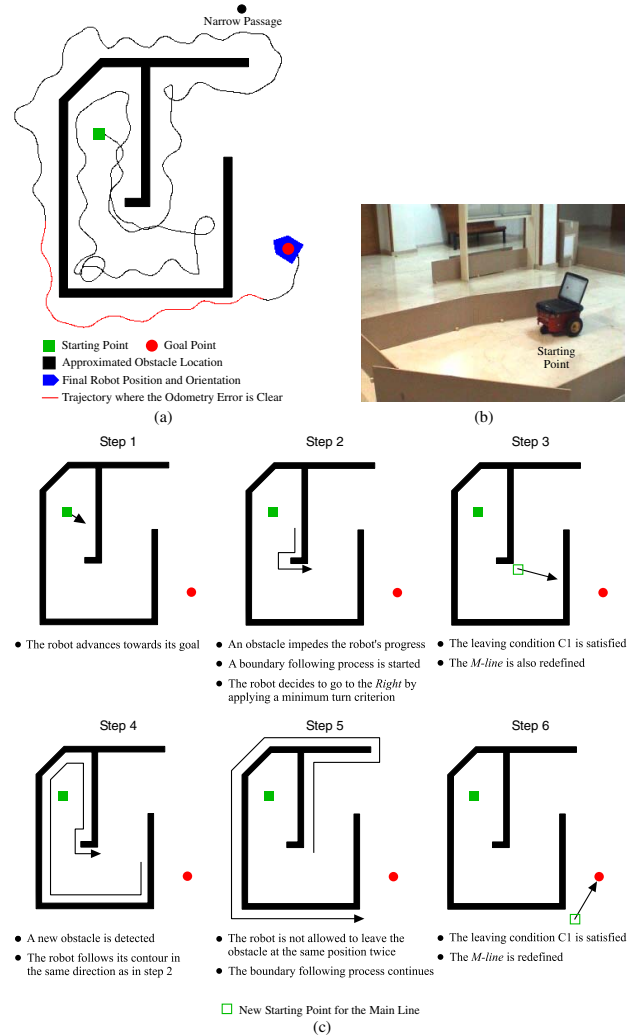


Fig. 8. Results for a real mission: (a) trajectory reconstructed with the odometry data; (b) a view of the environment; (c) the mission step by step.

Further experimentation with real robots as well as a comparative study on the path length performance of the proposal with regard to other five strategies selected from the related literature can be found in [6].

V. CONCLUSIONS

A novel reactive control strategy to avoid both local and global trapping situations has been presented and discussed in detail. It is called *Bug-based T^2* and is integrated in the framework of the classic artificial potential fields [5]. The convergence of the algorithm has been proved and its worst-case performance bounded for a restricted situation where, nevertheless, both convex and non-convex obstacles were considered.

Future research will tackle the problem of the oscillations suffered by the robot's trajectory while navigating (see fig. 8(a)), taking as a starting point the work presented in [3].

APPENDIX

Let S and T be two points representing, respectively, the initial robot position and the target of the mission in a two-dimensional navigation environment. Lemmas 1 and 2, and theorem 1 prove the convergence of the strategy *Bug-based T^2* in accordance with such notation.

Lemma 1:

Given a point S and a point T , the leaving condition C2 alone guarantees, whenever possible, reaching the target through a path P whose length never exceeds the limit

$$P = D + \sum_i \frac{n_i p_i}{2}, \quad (2)$$

where D represents the Euclidean distance between S and T (M -line's length), i denotes the number of obstacles intersecting a disc of radius D centered at T and, finally, n_i and p_i designate, respectively, the number of intersections with the M -line and the perimeter of the i th obstacle.

Assumption/s: *The robot knows its own coordinates and those of T all the time.*

Proof: *Refer to [4].*

Lemma 2:

The leaving condition C1 can only be satisfied a limited number of times.

Assumption/s: *The workspace is bounded. Consequently, the perimeter and the number of obstacles as well as the distance between each other are finite.*

Proof: *The leaving condition C1 is met when the robot faces T out of the context of a potential deadlock situation³ while following the obstacle contour. In this way, given an obstacle of arbitrary shape and*

³Remember that this particular situation is related to the detection of environment concavities (refer to section III-C for further information).

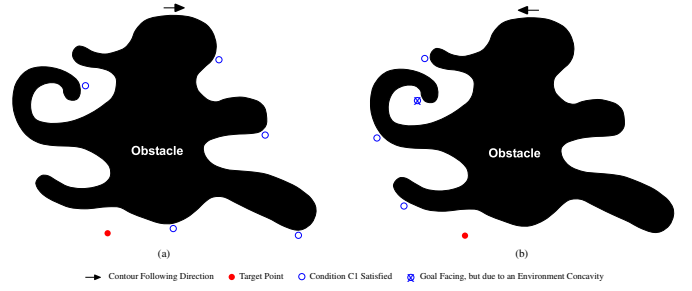


Fig. 9. Leaving points along the boundary of an obstacle with regard to condition C1, assuming that the contour following direction is left (a) or right (b). Additionally, note that such set of points was obtained by considering all possible starting robot locations (S).

finite perimeter, there will also be a finite number of places along the boundary of such obstacle where condition C1 will be satisfied (see figure 9 for an illustrative example). This fact in combination with the prohibition of abandoning the contour following process more than once in each of those places proves the lemma for the case of one obstacle. On the other hand, the proof for the general case is immediate by considering that the number of obstacles in the navigation environment is finite.

Theorem 1:

The strategy *Bug-based T^2* ensures convergence.

Assumption/s: *Those ones associated with lemmas 1 and 2.*

Proof: *Each time the leaving condition C1 is fulfilled, a new starting point is defined by setting S equal to the current robot location. From that moment, based on lemma 1, the algorithm would converge if condition C2 was exclusively considered or, in other words, if condition C1 was never satisfied again. In that last respect, it is known that such a situation will occur in finite period of time according to lemma 2.*

ACKNOWLEDGMENT

This study has been partially supported by project CICYT-DPI2005-09001-C03-02 and FEDER funds.

REFERENCES

- [1] J. Antich and A. Ortiz, "Extending the potential fields approach to avoid trapping situations," in *Proceedings of the Intl. Conference on Intelligent Robots and Systems*, 2005, pp. 1379–1384.
- [2] Y. Koren and J. Borenstein, "Potential field methods and their inherent limitations for mobile robot navigation," in *Proceedings of the Intl. Conference on Robotics and Automation*, 1991, pp. 1398–1404.
- [3] A. Masoud, "Solving the narrow corridor problem in potential field-guided autonomous robots," in *Proceedings of the Intl. Conference on Robotics and Automation*, 2005, pp. 2920–2925.
- [4] V. Lumelsky and A. Stepanov, "Path planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape," *Algorithmica*, vol. 2, pp. 403–430, 1987.
- [5] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Intl. Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [6] J. Antich, "Reactive robotics: A paradigm not limited to simple tasks," 2006, *Research Report*, University of the Balearic Islands.